

Q1(a)

A stack is a linear data structure in which insertion and deletion are done at one end only, called the top. It follows the LIFO principle - Last In First Out.



Applications: 1) undo operation in text editors 2) function call stack used in recursion.

Q1(b)

Stack follows LIFO whereas a queue follows FIFO (First In First out). In a stack insertion and deletion happen at the same end (top) but in a queue insertion is at the rear and deletion is at the front.

Example: a pile of plates in a cafeteria is a stack, people waiting in a line at a ticket counter form a queue.

Q2

Binary search is used to search a key in a SORTED array.

Precondition: the array must be sorted.

Algorithm: find the middle element. If  $\text{key} == \text{mid}$ , return. If  $\text{key} < \text{mid}$  search the left half, else search the right half. Repeat until found or the range is empty.

Complexity: each step halves the search space, so after  $k$  steps  $n/2^k$  elements are left. Search ends when  $n/2^k = 1$ , so  $k = \log_2 n$ .

The recurrence is  $T(n) = T(n/2) + O(1)$  which gives  $O(\log n)$  in the worst case. Best case is  $O(1)$ . Space is  $O(1)$  for the iterative version.

### Q3(a)

BFS algorithm:

1. Mark the source vertex visited and enqueue it.
  2. While the queue is not empty: dequeue a vertex  $u$  and visit it.
  3. For every unvisited neighbour  $v$  of  $u$ , mark  $v$  visited and enqueue it.
- Time complexity is  $O(V + E)$  using adjacency lists.

### Q3(b)

BFS uses a queue and explores level by level; DFS uses a stack (or recursion) and goes deep along a branch before backtracking.

Both take  $O(V + E)$  time.

Use cases: BFS gives shortest path in an unweighted graph; DFS is used for cycle detection, topological sort and connected components.

### Q4

To reverse a singly linked list we use three pointers  $prev$ ,  $curr$  and  $next$ . Initially  $prev = null$  and  $curr = head$ .

Loop while  $curr$  is not null:  $next = curr.next$ ;  $curr.next = prev$ ;  $prev = curr$ ;  $curr = next$ .

At the end  $prev$  is the new head.

The list is traversed once so the time complexity is  $O(n)$  and the extra space is  $O(1)$ .